

EstacionaTec Guia comentado do front-end

HTML, componentes TypeScript e localização das funcionalidades

Retrato do código disponível em 14 de setembro de 2026. Documento de estudo e consulta para o TCC, baseado nos arquivos locais do projeto estacionatec-web.

O sistema organiza a interface de controle de estacionamento: entrada, saída, cadastros, histórico, usuários, monitoramento, imagens e relatórios.

Como usar este guia

Cada tela apresenta sua finalidade, o papel do HTML, os métodos do componente, um trecho real comentado e as limitações existentes. As referências usam caminhos relativos à raiz do projeto e a linha inicial do trecho para facilitar a busca no editor.

Raiz analisada: C:/EstacionaTec/estacionatec-web

Importante para a apresentação

Este é um front-end com comportamentos locais e dados demonstrativos. Uma mensagem de sucesso na interface não comprova gravação em PostgreSQL, autenticação, leitura de câmera ou abertura física do portão. Essas diferenças são apontadas ao longo do documento.

Os comentários estão neste PDF: nenhum HTML ou componente da aplicação foi modificado para produzir o guia.

Roteiro de leitura

- 03 / Arquitetura e inicialização
- 04 / Linguagem dos templates Angular
- 05 / Layout, cabeçalho e navegação
- 06 / Serviço e dados compartilhados
- 07 / Login
- 08 / Recuperação de senha
- 09 / Dashboard
- 10 / Registrar entrada
- 11 / Registrar saída
- 12 / Veículos
- 13 / Pessoas
- 14 / Histórico
- 15 / Usuários
- 16 / Câmera ao vivo
- 17 / Imagens capturadas
- 18 / Relatórios
- 19 / Fluxos e pontos de atenção
- 20 / Testes, estilos e consulta rápida

Convenções

HTML define a estrutura visível e conecta eventos. TypeScript mantém o estado e executa as operações. CSS controla a apresentação. Um método é uma função definida na classe; um getter calcula um valor quando a propriedade é consultada.

Os números de linha se referem ao código desta data e podem mudar com futuras edições. Procure também pelo nome do método indicado.

Arquitetura e inicialização

O projeto utiliza Angular com NgModules. Os componentes declaram standalone: false e são registrados no AppModule. As versões declaradas no package.json são Angular ^22.1.0, TypeScript ~6.0.2 e RxJS ~7.8.0; os testes usam Vitest.

Sequência de carregamento

src/main.ts inicia AppModule; o módulo inicializa App; app.html exibe o router-outlet; o roteador escolhe a tela. Nas rotas internas, MainLayoutComponent contém outro router-outlet para a página selecionada.

src/main.ts | linha 4 | bootstrapModule

src/app/app-module.ts | linha 24 | declarations:

src/app/app.html | linha 1 | <router-outlet>

<router-outlet></router-outlet>

O componente raiz não desenha a sidebar nem o conteúdo das páginas. Ele reserva o local em que a rota será renderizada.

Responsabilidades do módulo

BrowserModule fornece recursos para execução no navegador; FormsModule habilita formulários com ngModel; AppRoutingModule disponibiliza a navegação. templateUrl associa a classe ao HTML e styleUrls associa o CSS local.

src/app/app-module.ts | linha 42 | imports: [BrowserModule

Rotas

Login e recuperar-senha ficam fora do layout interno. As demais telas são filhas do MainLayoutComponent. data.title fornece o título do cabeçalho. O primeiro caminho vazio redireciona para login; a rota ** redireciona endereços desconhecidos para dashboard. Não existem guards neste arquivo.

src/app/app-routing-module.ts | linha 17 | const routes

Organização

layout contém a estrutura visual; pages reúne as telas; core/services contém o cadastro compartilhado. A pasta footer existe, mas seus arquivos estão vazios e ela não está declarada no AppModule. Não há um rodapé funcional montado no layout atual.

Linguagem dos templates Angular

Interpolação: `{{ valor }}`

Lê uma propriedade ou expressão TypeScript e coloca o resultado no HTML. Exemplo: `{{ r.owner }}` mostra o proprietário do registro do histórico.

Ligação de propriedade: `[disabled]`

Envia um valor da classe para uma propriedade do elemento. Em Monitoramento, `[disabled]="refreshing"` desabilita o botão enquanto a atualização simulada está em andamento.

Evento: `(click)` e `(ngSubmit)`

`(click)` responde ao clique; `(ngSubmit)` responde ao envio do formulário. O botão Registrar Entrada envia o formulário que chama `submit()`. Botões auxiliares usam `type="button"` para não enviar o formulário.

Ligação em duas direções: `[(ngModel)]`

Mantém campo e variável sincronizados. Ao digitar uma placa, `plate` é atualizado. Quando o TypeScript normaliza `plate`, o campo também muda. Dentro de formulários, `name` identifica o controle.

Blocos `@if` e `@for`

`@if` decide se um trecho existe na tela. `@for` cria uma linha para cada item. `track` informa a chave de identificação do item, como `id`, `placa` ou `documento`; não é uma consulta ao banco.

Pipes e rotas

O pipe `date` formata datas para exibição, sem alterar o dado original. `routerLink` navega na aplicação; `routerLinkActive` destaca o menu; `queryParams` leva parâmetros na URL.

Estado, cópias e listas

`modal`, `pendingDelete` e `selectedImage` controlam blocos visuais. `{...objeto}` copia campos; `map` atualiza um item; `filter` seleciona ou remove itens; `find` procura o primeiro correspondente. Esses métodos operam nas listas em memória.

Layout, cabeçalho e navegação

MainLayoutComponent

sidebarOpen inicia como true. O HTML usa @if para montar a sidebar e passa esse estado ao Header. Ao receber toggleSidebar, o layout inverte o valor. O conteúdo da rota filha aparece abaixo do cabeçalho.

src/app/layout/main-layout/main-layout.component.html | linha 7 | <app-header

```
<app-header
  [sidebarOpen]="sidebarOpen"
  (toggleSidebar)="sidebarOpen = !sidebarOpen"
></app-header>
```

[sidebarOpen] é a entrada de dados do cabeçalho; (toggleSidebar) é o evento emitido pelo filho e tratado pelo layout pai.

src/app/layout/main-layout/main-layout.component.ts | linha 10 | sidebarOpen = true

HeaderComponent

@Input recebe sidebarOpen e @Output expõe um EventEmitter. O botão em header.component.html chama toggleSidebar.emit(). A classe escuta NavigationEnd, percorre a rota até a última filha e consulta data.title. takeUntilDestroyed encerra a inscrição quando o componente é destruído.

src/app/layout/header/header.component.ts | linha 13 | @Input()

src/app/layout/header/header.component.ts | linha 27 | this.router.events

A data usa Intl.DateTimeFormat com pt-BR. Ela é calculada ao criar o cabeçalho; não é um relógio atualizado continuamente.

SidebarComponent

A classe não tem métodos porque o comportamento está nas diretivas do template. Os links apontam para as telas e recebem active na rota selecionada. Logo1.png vem de public/assets. O nome João Carlos e o nível Administrador são fixos no HTML.

src/app/layout/sidebar/sidebar.component.html | linha 6 | routerLink="/dashboard"

Sair apenas navega para /login. Não remove token nem encerra sessão no servidor, pois não há autenticação real implementada.

src/app/layout/sidebar/sidebar.component.html | linha 26 | class="logout"

Serviço e dados compartilhados

ParkingRegistryService é o ponto compartilhado por Pessoas, Veículos e Entrada. providedIn: root disponibiliza uma instância para a aplicação. Não é um cliente HTTP e não representa um banco persistente.

Modelos de dados

RegisteredVehicle contém id, placa, modelo, cor, proprietário e tipo do veículo. RegisteredPerson contém nome, documento, e-mail, telefone e categoria da pessoa. EntryRecord guarda uma fotografia dos dados da entrada e enteredAt como Date.

src/app/core/services/parking-registry.service.ts | linha 3 | export interface RegisteredVehicle

src/app/core/services/parking-registry.service.ts | linha 20 | export interface EntryRecord

Normalização e registro

src/app/core/services/parking-registry.service.ts | linha 30 | export function normalizePlate

```
export function normalizePlate(value: string): string {  
  return value.trim().toUpperCase().replace('-', '');  
}
```

trim remove espaços nas extremidades; toUpperCase padroniza letras; replace remove o hífen. ABC-1234 e abc1234 passam a ter a mesma chave de comparação.

registerEntry valida três letras e quatro números, procura o veículo e depois o proprietário pelo nome. Se não encontrar, lança Error. Quando encontra, monta EntryRecord, usa new Date() e adiciona o registro a entries.

src/app/core/services/parking-registry.service.ts | linha 87 | registerEntry(value

O que os dados atuais não garantem

A expressão aceita o padrão antigo ABC-1234, mas não placas Mercosul como ABC1D23. Não há verificação de uma entrada já aberta, capacidade ou regras por horário. O vínculo pessoa-veículo é textual pelo nome, sujeito a homônimos.

people, vehicles e entries desaparecem ao recarregar a aplicação. readonly em entries impede trocar a referência, mas não impede push. Saída e Histórico não consomem entries, portanto ainda não fecham o mesmo ciclo de estacionamento.

Login

Rota /login. Tela inicial para apresentar o acesso de administradores e porteiros.

HTML: o que aparece e como reage

O formulário liga email e password com ngModel e chama login(). showPassword alterna o tipo do campo entre text e password. @if mostra mensagens de erro; routerLink leva à recuperação de senha.

src/app/pages/auth/login/login.component.html | linha 7 | (ngSubmit)

TypeScript: onde está a funcionalidade

login() verifica se o usuário foi preenchido e se a senha tem pelo menos seis caracteres. Se não houver erro, Router.navigate abre /dashboard.

src/app/pages/auth/login/login.component.ts | linha 16 | login():

Trecho comentado

src/app/pages/auth/login/login.component.ts | linha 16 | login():

```
login(): void {  
  this.error =  
    !this.email || this.password.length < 6  
      ? 'Informe o usuário e uma senha com pelo menos 6 caracteres.'  
      : '';  
  if (!this.error) this.router.navigate(['/dashboard']);  
}
```

O teste é de preenchimento e comprimento, não de autenticidade. O navigate altera a rota; não comprova login em um servidor.

Limites atuais

Não consulta usuários cadastrados, não compara senha, não cria sessão e não aplica níveis de acesso. As credenciais exibidas são demonstrativas; qualquer usuário não vazio e senha com tamanho suficiente passam na verificação atual.

Recuperação de senha

Rota /recuperar-senha. Interface para solicitar instruções de recuperação.

HTML: o que aparece e como reage

@if (!sent) mostra o formulário; o bloco alternativo mostra a confirmação. O campo de e-mail usa ngModel; o envio chama send(). O CSS reutiliza o arquivo da tela de login.

src/app/pages/auth/recuperar-senha/recuperar-senha.component.html | linha 4 | @if (!sent)

TypeScript: onde está a funcionalidade

send() testa o e-mail com uma expressão regular simples e, quando válido, muda sent para true.

src/app/pages/auth/recuperar-senha/recuperar-senha.component.ts | linha 12 | send():

Trecho comentado

src/app/pages/auth/recuperar-senha/recuperar-senha.component.ts | linha 12 | send():

```
send(): void {  
  this.error = /^[^\\s@]+@[^\\s@]+\\.([^\\s@]+)$/.test(this.email) ? '' : 'Informe um e-mail  
válido.';  
  if (!this.error) this.sent = true;  
}
```

Alterar sent muda apenas qual bloco do HTML é renderizado. A expressão regular valida formato básico, não a existência da conta.

Limites atuais

Nenhum e-mail é enviado. A mensagem de envio e a expiração em 24 horas são texto de demonstração, sem token, prazo real ou integração de recuperação.

Dashboard

Rota /dashboard. Visão geral com três indicadores e tabela de veículos estacionados.

HTML: o que aparece e como reage

O primeiro `@for` percorre `stats` para construir os cartões. O segundo percorre `vehicles` e usa índices: `v[0]` placa, `v[1]` proprietário, `v[2]` categoria e `v[3]` entrada. A etiqueta `Ativo` é escrita diretamente no HTML.

`src/app/pages/dashboard/dashboard.component.html` | linha 3 | `@for (stat`

TypeScript: onde está a funcionalidade

`stats` define os valores 5 veículos, 9 entradas e 4 saídas. `vehicles` contém cinco linhas fixas. A classe não possui métodos de cálculo ou atualização.

`src/app/pages/dashboard/dashboard.component.ts` | linha 8 | `stats =`

Trecho comentado

`src/app/pages/dashboard/dashboard.component.ts` | linha 8 | `stats =`

```
stats = [  
  ['Veículos Estacionados', '5'],  
  ['Entradas Hoje', '9'],  
  ['Saídas Hoje', '4'],  
];
```

Os números são strings fixas. Para alterar o demonstrativo, o local é `stats`; para mostrar totais reais, será necessário fornecer dados e cálculos integrados.

Limites atuais

Apesar do texto “em tempo real”, a tela não recebe eventos de Entrada ou Saída. Os totais não são calculados a partir dos registros; representam o protótipo.

Registrar entrada

Rota /entrada. O operador informa somente a placa para consultar veículo e proprietário e registrar o horário local.

HTML: o que aparece e como reage

O formulário chama submit(); ngOnModelChange limpa o resultado anterior. O botão Abrir Portão chama openGate(). @if (entry) mostra placa, proprietário, categoria, modelo e hora; @if (error) mostra a falha.

src/app/pages/entrada/entrada.component.html | linha 13 | (ngSubmit)

TypeScript: onde está a funcionalidade

submit() limpa mensagens, chama registry.registerEntry(plate), recebe EntryRecord e substitui a placa pelo formato padronizado. try/catch converte erros em mensagens.

src/app/pages/entrada/entrada.component.ts | linha 22 | submit():

openGate() informa que a abertura manual está indisponível, sem acionar hardware.

src/app/pages/entrada/entrada.component.ts | linha 33 | openGate():

Trecho comentado

src/app/pages/entrada/entrada.component.ts | linha 26 | this.entry = this.registry

```
this.entry = this.registry.registerEntry(this.plate);  
this.plate = this.entry.plate;  
} catch (error) {
```

A regra de validação e consulta está no serviço; o componente cuida do formulário e do retorno visual. O catch impede que uma falha de consulta seja apresentada como sucesso.

Limites atuais

A busca ocorre ao enviar o formulário, não automaticamente a cada tecla. O horário usa o relógio do navegador. A entrada é adicionada ao serviço em memória e não atualiza as listas independentes de Saída/Histórico.

Registrar saída

Rota /saida. Busca e retirada de um veículo da lista demonstrativa de estacionados.

HTML: o que aparece e como reage

O campo atualiza search. @for percorre filtered e apresenta placa, proprietário, marca, modelo e entrada. O botão chama exit(v.plate). confirmed controla a mensagem de sucesso e a lista vazia mostra um aviso.

src/app/pages/saida/saida.component.html | linha 49 | (click)="exit

TypeScript: onde está a funcionalidade

filtered concatena placa, proprietário, marca, modelo e hora de entrada para uma busca textual sem diferenciar maiúsculas e minúsculas.

src/app/pages/saida/saida.component.ts | linha 17 | get filtered()

exit() salva a placa em confirmed e usa filter para removê-la de vehicles.

src/app/pages/saida/saida.component.ts | linha 25 | exit(plate:

Trecho comentado

src/app/pages/saida/saida.component.ts | linha 25 | exit(plate:

```
exit(plate: string): void {  
  this.confirmed = plate;  
  this.vehicles = this.vehicles.filter((v) => v.plate !== plate);  
}
```

A saída atual significa remover uma linha da lista desta tela. A mensagem de sucesso não representa encerramento persistido de uma permanência.

Limites atuais

Não calcula nem grava data/hora da saída. Não atualiza Histórico, Dashboard ou entries do serviço. Ao recriar a página, a lista local começa novamente com os exemplos.

Veículos

Rota /veiculos. Cadastro, busca, edição e exclusão de veículos; exibe também a categoria do proprietário.

HTML: o que aparece e como reage

@for usa v.id para as linhas. Editar chama edit(v); Excluir preenche pendingDelete e abre a confirmação. O formulário usa form e chama save(). category(v) preenche a coluna da categoria da pessoa.

src/app/pages/veiculos/veiculos.component.html | linha 90 | (ngSubmit)

TypeScript: onde está a funcionalidade

category() consulta people pelo nome do proprietário. Se não houver correspondência, retorna Não identificado.

src/app/pages/veiculos/veiculos.component.ts | linha 29 | category(vehicle:

edit() guarda editingId e copia os campos, evitando alterar a linha antes de Salvar. closeModal() descarta o formulário.

src/app/pages/veiculos/veiculos.component.ts | linha 44 | edit(vehicle:

save() atualiza por id ou cria com maior id + 1. confirmDelete() remove o id selecionado do serviço.

src/app/pages/veiculos/veiculos.component.ts | linha 63 | save()

Trecho comentado

src/app/pages/veiculos/veiculos.component.ts | linha 72 | const nextId

```
const nextId = Math.max(0, ...this.vehicles.map((vehicle) => vehicle.id)) + 1;  
this.vehicles = [...this.vehicles, { id: nextId, ...this.form }];
```

O spread preserva os itens existentes e acrescenta o novo registro. O id é local, sem geração pelo banco.

Limites atuais

A validação exige apenas placa e proprietário preenchidos; não verifica duplicidade, padrão da placa ou existência do proprietário no cadastro. O tipo Carro/Moto é diferente da categoria Aluno/Professor/Funcionário.

Pessoas

Rota /pessoas. Gerencia pessoas e mostra informações do proprietário acessado pelo Histórico.

HTML: o que aparece e como reage

A tabela lista nome, documento, contato e categoria. Editar e Excluir abrem os respectivos fluxos. Quando ownerDocument está presente, um cartão de consulta exibe os detalhes ou uma mensagem de cadastro não encontrado.

src/app/pages/pessoas/pessoas.component.html | linha 9 | @if (ownerDocument)

TypeScript: onde está a funcionalidade

ngOnInit() acompanha proprietario na URL. selectedOwner procura o documento no serviço; takeUntilDestroyed encerra a inscrição.

src/app/pages/pessoas/pessoas.component.ts | linha 19 | ngOnInit():

save() exige nome/documento e impede documento repetido. Ao renomear, atualiza os veículos vinculados pelo nome anterior.

src/app/pages/pessoas/pessoas.component.ts | linha 82 | save():

confirmDelete() impede excluir quem ainda possui veículos. edit() usa cópia; closeModal() cancela sem salvar.

src/app/pages/pessoas/pessoas.component.ts | linha 66 | confirmDelete():

Trecho comentado

src/app/pages/pessoas/pessoas.component.ts | linha 15 | get selectedOwner()

```
get selectedOwner(): RegisteredPerson | undefined {  
  return this.people.find((person) => person.document === this.ownerDocument);  
}
```

O cartão procura a pessoa atual pelo documento recebido. Não cria cadastro e não abre automaticamente o formulário de edição.

Limites atuais

Não valida os dígitos verificadores do CPF. O vínculo por nome pode ser ambíguo. Trocar o documento pode invalidar links antigos do Histórico, que mantém documentos fixos.

Histórico

Rota /historico. Consulta apenas permanências encerradas, com modelo, entrada, saída e tempo estacionado.

HTML: o que aparece e como reage

O campo busca placa ou proprietário. A tabela aplica o pipe date para data/hora, chama duration(r) e usa routerLink /pessoas com queryParams para o botão Ver proprietário. Ausência de modelo/documento gera avisos.

src/app/pages/historico/historico.component.html | linha 49 | [queryParams]

TypeScript: onde está a funcionalidade

filtered exige uma saída preenchida e aplica a busca textual. Isso oculta o registro de Carlos, que tem exit: null.

src/app/pages/historico/historico.component.ts | linha 58 | get filtered()

duration() subtrai os instantes, converte milissegundos para minutos e mostra horas/minutos. Datas inválidas ou duração negativa têm retorno específico.

src/app/pages/historico/historico.component.ts | linha 66 | duration(record:

Trecho comentado

src/app/pages/historico/historico.component.ts | linha 70 | const minutes =

```
const minutes = Math.floor(elapsed / 60000);  
const hours = Math.floor(minutes / 60);  
return hours > 0 ? `${hours}h ${minutes % 60}min` : `${minutes}min`;
```

60000 converte milissegundos em minutos; Math.floor descarta segundos; % 60 obtém os minutos restantes. Permanências de mais de um dia continuam em horas totais.

Limites atuais

records é uma lista própria: registrar saída em outra tela não insere uma linha aqui. Modelo e documento são fotografias demonstrativas. A coluna Modelo existe, mas não participa da busca atual. O documento vai na URL, não é um id técnico.

Usuários

Rota /usuarios. Gerencia os cadastros demonstrativos de administradores e porteiros.

HTML: o que aparece e como reage

A tabela mostra Ações com Editar e Excluir. O formulário muda título e botão conforme editingUser. Senha aparece apenas na criação. Há seleção de nível/status, aviso de erro e confirmação separada para excluir.

src/app/pages/usuarios/usuarios.component.html | linha 45 | (click)="edit(u)

TypeScript: onde está a funcionalidade

edit() guarda a referência do cadastro e copia os campos. save() exige nome, usuário e e-mail; bloqueia username repetido ignorando maiúsculas e exige seis caracteres de senha só na criação.

src/app/pages/usuarios/usuarios.component.ts | linha 63 | edit(user:

confirmDelete() remove o objeto escolhido. administrators e gatekeepers contam os papéis, inclusive quando a lista fica vazia.

src/app/pages/usuarios/usuarios.component.ts | linha 84 | confirmDelete():

Trecho comentado

src/app/pages/usuarios/usuarios.component.ts | linha 115 | this.users = this.editingUser

```
this.users = this.editingUser
  ? this.users.map((item) => (item === this.editingUser ? user : item))
  : [...this.users, user];
this.closeModal();
```

Na edição, map substitui somente o objeto original; na criação, o spread acrescenta um usuário. closeModal limpa o estado para a próxima operação.

Limites atuais

users é local ao componente. Senhas são removidas antes de armazenar o objeto; não existe autenticação integrada. Alterar nível/status não aplica permissões. Não há regra impedindo excluir o último administrador.

Câmera ao vivo

Rota /monitoramento. Painel demonstrativo com relógio, controles e lista de eventos.

HTML: o que aparece e como reage

#cameraPanel identifica o elemento para tela cheia. Botões chamam refresh(), capture() e fullscreen(cameraPanel). selectedEvent abre um modal com os dados do evento. Não existe elemento de vídeo transmitindo imagem.

src/app/pages/monitoramento/monitoramento.component.html | linha 3 | #cameraPanel

TypeScript: onde está a funcionalidade

O intervalo atualiza currentTime a cada segundo. markForCheck solicita atualização da interface. refresh() usa um atraso de 1,5 segundo e depois mostra aviso.

src/app/pages/monitoramento/monitoramento.component.ts | linha 22 | private readonly clock

fullscreen() usa a API do navegador e trata rejeição. capture() somente informa indisponibilidade; ngOnDestroy() cancela intervalo e timeout.

src/app/pages/monitoramento/monitoramento.component.ts | linha 43 | async fullscreen

Trecho comentado

src/app/pages/monitoramento/monitoramento.component.ts | linha 53 | ngOnDestroy():

```
ngOnDestroy(): void {  
  clearInterval(this.clock);  
  clearTimeout(this.refreshTimer);  
}
```

Limpar temporizadores ao sair evita que o relógio e a atualização continuem executando após a destruição do componente.

Limites atuais

Tela cheia é uma ação real do navegador, mas câmera, reconhecimento de placa, captura e contadores 125/3 são demonstrativos. Os eventos não vêm de hardware nem do serviço de entradas.

Imagens capturadas

Rota /imagens. Consulta de metadados de capturas demonstrativas, com filtros e detalhes.

HTML: o que aparece e como reage

Quatro controles filtram placa, data, tipo e status; ngOnModelChange chama resetPage(). @for lista visibleImages. Visualizar define selectedImage e abre o modal. Os botões de paginação respeitam os limites.

src/app/pages/imagens/imagens.component.html | linha 12 | (ngModelChange)="resetPage()"

TypeScript: onde está a funcionalidade

filtered combina todos os filtros com &&. Uma opção vazia não restringe o resultado. A busca por placa usa includes, sem normalizar o hífen.

src/app/pages/imagens/imagens.component.ts | linha 73 | get filtered():

totalPages calcula o total; visibleImages usa slice com pageSize 10. changePage limita o intervalo e dateTime formata o texto de data/hora.

src/app/pages/imagens/imagens.component.ts | linha 83 | get totalPages():

download() informa que o arquivo é demonstrativo; não transfere imagem.

src/app/pages/imagens/imagens.component.ts | linha 98 | download(image:

Trecho comentado

src/app/pages/imagens/imagens.component.ts | linha 86 | get visibleImages():

```
get visibleImages(): CapturedImage[] {  
  return this.filtered.slice((this.page - 1) * this.pageSize, this.page * this.pageSize);  
}
```

slice seleciona somente os itens da página atual depois dos filtros. A paginação ocorre no navegador, não em uma API.

Limites atuais

São quatro registros de 07/05/2026, portanto inicialmente existe só uma página. imagePath é texto gerado, não prova de arquivo existente. Miniatura e prévia são placeholders, não fotografias.

Relatórios

Rota /relatorios. Interface para consultar exemplos e selecionar um período de relatório.

HTML: o que aparece e como reage

@for lista recentReports. O formulário liga start, end e type; ngSubmit chama generate(). Baixar chama download(). A propriedade message aparece como aviso na tela.

src/app/pages/relatorios/relatorios.component.html | linha 19 | (ngSubmit)

TypeScript: onde está a funcionalidade

generate() verifica ausência de datas e início posterior ao fim. Períodos válidos recebem o aviso de integração futura.

src/app/pages/relatorios/relatorios.component.ts | linha 17 | generate()

download() informa que o PDF demonstrativo não está disponível. recentReports guarda nomes, datas e tamanhos fixos.

src/app/pages/relatorios/relatorios.component.ts | linha 23 | download():

Trecho comentado

src/app/pages/relatorios/relatorios.component.ts | linha 17 | generate()

```
generate() {  
  this.message =  
    !this.start || !this.end || this.start > this.end  
    ? 'Informe um período válido para o relatório.'  
    : 'A geração de relatórios estará disponível após a integração com o servidor.';  
}
```

Datas de input type=date chegam em formato ano-mês-dia. A comparação textual funciona para esse formato padronizado; ela não consulta movimentações.

Limites atuais

Nenhum relatório é gerado ou baixado pela aplicação. type registra a escolha, mas não seleciona uma rotina de cálculo. O PDF deste guia é documentação externa, não uma implementação do botão de relatórios.

Fluxos e pontos de atenção

Fluxo já compartilhado

Pessoas.save() altera registry.people. Veículos.save() altera registry.vehicles. Entrada.submit() chama registerEntry(), que consulta essas listas e preenche entries. Os dados sobrevivem à troca de rota por estarem no serviço raiz, mas não ao recarregamento.

Fluxo de consulta entre telas

Histórico monta /pessoas?proprietario=...; Pessoas.ngOnInit() lê o parâmetro; selectedOwner localiza o documento; o HTML apresenta o cartão. O restante da listagem de Pessoas permanece disponível. Voltar ao histórico faz nova navegação, sem preservar a busca anterior.

Fluxos que ainda estão separados

Saída.exit() remove da lista local. Histórico.records e Dashboard.vehicles/stats não são atualizados. Portanto, ainda não há uma única permanência compartilhada com entrada, saída, duração e ocupação sincronizadas.

Melhorias futuras, não implementadas neste guia

1. Centralizar movimentações com identificadores estáveis, horário de entrada e saída; alimentar Dashboard, Saída e Histórico a partir da mesma fonte.
2. Vincular veículo e proprietário por id, em vez de nome; usar id técnico nas URLs no lugar do documento pessoal e tratar alterações/exclusões mantendo o histórico.
3. Integrar o backend, persistência e autorização. Validações visuais não substituem regras no servidor. A situação atual permite acessar rotas internas diretamente.
4. Unificar a modelagem de marca/modelo e a normalização de placas; definir suporte a Mercosul e prevenção de entradas duplicadas conforme os requisitos.
5. Integrar câmera, reconhecimento, portão e geração real de relatórios somente quando os serviços correspondentes estiverem disponíveis.

Testes, estilos e consulta rápida

Como conferir o comportamento

npm start inicia o desenvolvimento; npm run build compila; npm test -- --watch=false executa os testes uma vez. A última execução registrada nesta tarefa passou com 35 testes em oito arquivos e build concluído. Esse resultado é da implementação anterior ao guia, não uma nova execução para este PDF.

Os arquivos *.component.spec.ts documentam cenários executáveis: Entrada (9), Pessoas (7), Usuários (6), Histórico (4), Veículos (4), Imagens (2), Monitoramento (1), além de app.spec.ts (2). Não há cobertura completa de todas as telas, nem testes reais de backend ou hardware.

src/app/pages/historico/historico.component.spec.ts | linha 7 | describe(

src/app/pages/usuarios/usuarios.component.spec.ts | linha 5 | describe(

Onde mudar a aparência

src/styles.css define card, btn, input, table, badge, modal e grades responsivas usadas em várias telas. Apesar dos nomes parecidos, essas classes são CSS próprio; Bootstrap não está declarado nas dependências do projeto. Entrada, Monitoramento, Imagens, Login e layout também possuem CSS local.

src/styles.css | linha 183 | .modal-backdrop

angular.json | linha 37 | "styles"

Mapa rápido de manutenção

Navegação: app-routing-module.ts. Declarações: app-module.ts. Menu: layout/sidebar/sidebar.component.html. Título: layout/header/header.component.ts. Regras da entrada: core/services/parking-registry.service.ts. Formulários e ações: pages/<tela>/<tela>.component.ts. Colunas e botões: HTML correspondente.

Roteiro sugerido de estudo

Leia primeiro o HTML de uma tela, encontre o evento do botão e abra o método citado no TypeScript. Identifique qual lista ele altera. Depois confira se essa lista é local ou pertence ao serviço e leia o teste correspondente. Esse percurso explica tanto a interface quanto seus limites reais.

Fonte: arquivos locais do front-end inspecionados em 14/09/2026. Este guia não é uma auditoria de conformidade com o documento de requisitos nem uma descrição do backend futuro.